

SAD Assignment 1: Problems with Exploratory Methods and How Agile Lifecycle Model Improves GUI Development

By Gautham Dhinakar, CSFT, 230958088, Roll number: 44

Key Problems with Exploratory Methods

Exploratory software development methods come with several problems which impacts project success and the quality of the user interface:

➔ Lack of Structure and Documentation

Exploratory methods usually lack comprehensive documentation and structured planning, hence making it difficult to track progress, reproduce issues, and maintain consistency across development cycles. This absence of formal documentation creates confusion, which makes it difficult for team members to understand decisions made during different phases.

➔ Inconsistency in Results

The quality of the application obtained using exploratory methods heavily depend on individual tester skills and experience. Different team members may approach developing the same application differently, leading to inconsistency in results, making it difficult to ensure proper testing and quality assurance.

➔ Scalability Issues

Exploratory development becomes exponentially more difficult with the increase in the size of the project. The difficulty of problems grows dramatically with complexity, making exploratory methods unsuitable for large-scale software development projects since scaling becomes hard.

➔ Difficulty in Evaluating Effectiveness

Evaluating the success and coverage of exploratory methods is challenging. It becomes difficult to measure test coverage, track defects found, or assess overall quality improvement, making it hard for managers to evaluate project progress and allocate resources effectively.

➔ Time Management Challenges

Without proper structure and time management, exploratory sessions can extend beyond their intended scope, leading to inefficiencies and project delays.

How Agile Lifecycle Model Enhances GUI Development

The Agile lifecycle model addresses the problems faced with exploratory development while providing advantages for GUI and user interface development:

➔ Structured Iterative Development

The Agile lifecycle model breaks down GUI development into sprints (typically 1-4 weeks), each delivering functional interface components. This structure provides the benefits of exploration while also maintaining control over the project.

➔ Continuous User Feedback Integration

Unlike exploratory methods that may lack user input, the Agile lifecycle model emphasizes continuous collaboration with customers and stakeholders throughout the development process which ensures that the GUI model remains aligned with user needs and expectations.

➔ **Enhanced Collaboration**

The Agile lifecycle model promotes more efficiently functioning teams where designers, developers, and testers work closely together. This collaboration ensures that GUI development benefits from diverse perspectives and expertise.

➔ **Early Issue Detection and Resolution**

Through regular testing and feedback cycles, the Agile lifecycle model enables early detection of issues and bugs in the GUI. This aids in avoiding costly redesigns later in the development process.

➔ **Flexible Structure**

The Agile lifecycle model maintains enough structure through sprint planning, daily stand-ups, and retrospectives to ensure projects stay on track and meet deadlines.

Real-World Example: Mobile Banking App Development

This example demonstrates how the Agile lifecycle model can be used for GUI development:

➔ **Sprint 1 (2-3 weeks): Core Login Interface**

- Develop basic login screen with username/password fields
- Implement secure authentication
- Test with initial user group for usability feedback
- Deliver working login functionality

➔ **Sprint 2 (2-3 weeks): Account Overview GUI**

- Create dashboard showing account balances
- Design intuitive navigation between accounts
- Implement responsive design for different screen sizes
- Gather user feedback on information presentation

➔ **Sprint 3 (2-3 weeks): Transaction Interface**

- Build money transfer functionality
- Design clear transaction forms with validation
- Add transaction history view
- Test with real users for workflow efficiency

➔ **Sprint 4 (2-3 weeks): Enhanced Features**

- Add bill payment interface
- Implement advanced security features (2FA)
- Refine UI based on accumulated user feedback
- Optimize performance and accessibility

Key Advantages of the Agile Lifecycle model in this example

➔ **Faster Time-to-Market**

Users can access basic banking functions (login, balance checking) while additional features are still being developed, providing immediate value.

➔ **Continuous Quality Improvement**

Each sprint includes testing and user feedback, ensuring GUI usability issues are identified and resolved quickly rather than discovered at project end.

➔ **Adaptability to Change**

If regulatory requirements change (like adding 2FA), the team can adjust priorities in the next sprint without disrupting the entire project timeline.

➔ **User-Centric Design**

Regular user involvement ensures the GUI meets actual user needs rather than what the developer assumes, resulting in higher user satisfaction.

➔ **Risk Mitigation**

By delivering working software incrementally, teams reduce the risk of complete project failure and can pivot quickly if certain GUI approaches aren't working effectively.

Implementing the Agile lifecycle model transforms GUI development from an unpredictable exploratory process into a structured, user-focused methodology that delivers better interfaces more efficiently while maintaining the flexibility to adapt to changing requirements and user feedback.